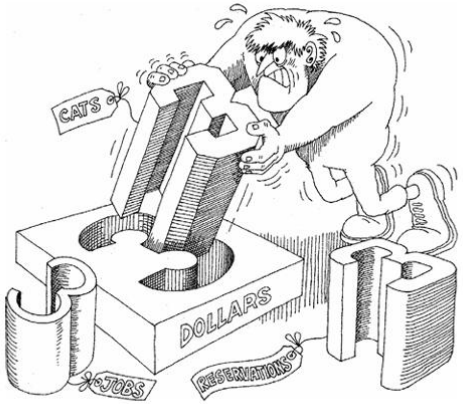
Standard Template Library (STL)

- STL was added to C++ by **Alexander Stepanov**
- Generic and efficient implementation of difficult-to-implement data structures
  - Linked list, Priority queue, Map
- Avoid errors with pointers, e.g. memory leaks, access violations
- Generic algorithms, e.g. sorting
- Generic classes such as containers and iterators
- Save effort on repeated implementations



Alexander Stepanov

Alexander Stepanov - Wikipedia



Strong typing prevents mixing of abstractions.

Image source: Object-Oriented Analysis and Design with Applications by Grady Booch et al.

Best Practices

- *Ordinarily, it is best to use vector unless there is a good reason to prefer another container.*
- If you are not sure which container to use, **write your code so that it uses only operations common to both vectors and lists**.
- Use iterators, not subscripts, and avoid random access to elements.
- That way it will be easy to use either a vector or a list as necessary.

Type Casting

Type Conversion

- Using construction, e.g. double d1 = double{2};
- OOPs/narrowing.cpp at main · ahanapradhan/OOPs
- C-style casting, e.g. int i = (int) ‘h’; or int i = int(‘h’);
- Static casting
  - Compile time check whether cast is correct
  - Conversions between related types, e.g. pointers of related classes, int <--> float
  - Narrowing conversions, Conversion to void *
- Dynamic casting
  - Runtime check whether conversion is safe
  - Used only with pointers / references
- Const casting
  - To convert const to a non-const
- Reinterpret casting
  - One pointer type to another unrelated type

Static Casting

```
int main() {
      int i ;
      float f = 3.14;
      i = static_cast<int>(f); // Same as i = int(f);
      char str[ ] = ”Delhi" ;
      cout << str ; // prints “Delhi”
      void *vptr ;
      vptr = static_cast<void*>(str);
      cout << vptr ; // prints an address
      Sample *sptr;
      Example *eptr ;
      sptr = (Sample *) eptr; // OK
      sptr = static_cast <Sample *> ( eptr ) ; // only allowed in related classes
}
```

const Casting

```
#include <iostream>
using namespace std ;
void display ( char * ) ;
int main( ) {
        const char *ptr = "What's up?";
        display(const_cast <char*>(ptr)); /* Usually a bad idea – unless you are
                                             interacting with old code */

        return 0 ;
}
void display ( char *p ) {
        cout << p << endl;
}
```

Passing C-style Strings

```
void fun1 ( char *p ) ;
void fun2 ( const char *p ) ;
int main( ) {
        char s1[ ] = "Hello" ; // s1 is NOT const
        const char *s2 = "Hello" ;
        fun1(s1); /* s1 is declared as char *, so
             allowed */
        fun1(s2); /* s2 is declared as const  char
        *, so NOT allowed */
        fun1("ABC"); /* string literals are      const
             char *, so NOT allowed */
        fun2(s1); // allowed
        fun2(s2); // allowed
        fun2("ABC") ; // allowed
}
```

```
void fun1 ( char *p ) {
}
void fun2 ( const char *p ) {
}
```

- Not allowed: const char * to char *
- Allowed: char * to const char *

Reinterpret Casting

```cpp
#include <iostream>
using namespace std;

void write(const char *p) {
    cout << *p << endl;
}

struct Book {
    const char *name;
    int publication_year;
    float price;
};

struct Employee {
    const char *name;
    int age;
    float salary;
};

int main() {
    Book b = {
        "Let us C++",
        1998,
        198.0f
    };

    Employee e = {
        "Ahana",
        35,
        0.0f
    };

    write(reinterpret_cast<const char *>(&b));
    write(reinterpret_cast<const char *>(&e));

    return 0;
}
```

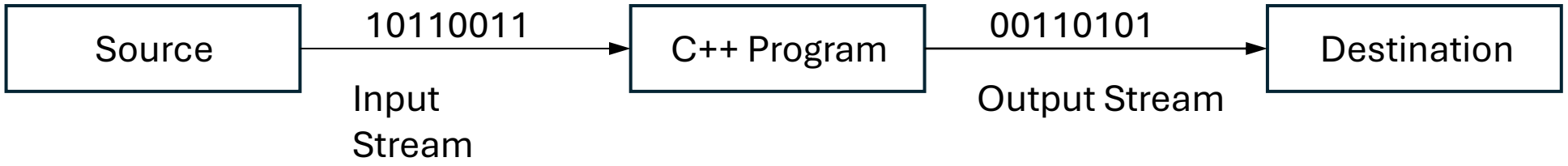
Summary

Cast type	Checked?	When to use
`T{expr}` (construction)	Compile time	Preferred, safe conversions
`(T)expr` (C-style)	None	Legacy code only
`static_cast`	Compile time	Safe, related conversions (int↔float, upcast)
`dynamic_cast`	Runtime	Safe downcasting with polymorphism
`const_cast`	Compile time	Remove/add const qualifiers
`reinterpret_cast`	None	Low-level bit reinterpretation

C++ Input and output

C++ Input and Output (I/O)

- Requirements
  - Perform I/O of char, numbers, strings, objects
  - Perform I/O from / to Keyboard, Screen, Network, Printer/Scanner
- Solution: Perform all I/O using Streams
- What is a Stream?
  - It is a sequence of bytes that travel from source to destination over a communication path

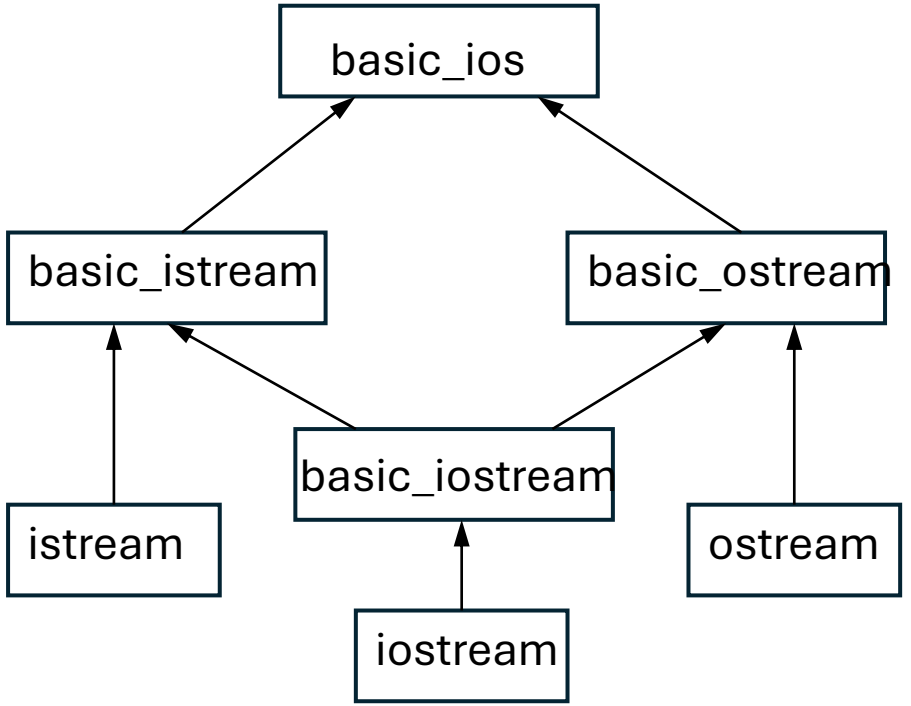


Stream-based IO

- To perform I/O C++ program can
  - Read from a stream or Write to a stream
- Linking of Streams to physical devices
  - Done by C++ I/O system and OS
- Streams are implemented by
  - Classes in std namespace - IOStream Library
- Benefits of using Streams
  - Streams hide details of communication from you
  - ***Methods are same, implementation changes as per device***

IOstream Library

- Several classes
  - basic_ios - Root class
- Console I/O
  - istream cin ;
  - ostream cout ;
- File I/O
  - ifstream - reading from file
  - ofstream - writing to file
  - fstream - reading / writing from file



Predefined streams

- System class contains
  - Predefined public static objects - cin, cout, cerr, clog
  - They are accessible from any part of the program
- Purpose
  - cin refers to standard input stream (default is keyboard)
  - cout refers to standard output stream (default is screen)
  - cerr refers to standard error stream (default is screen)
  - clog refers to standard log stream (default is screen)
- Specialty
  - These streams are already open when a program starts execution
  - They are ready to receive / send input / output data
  - Each of these can be redirected using ‘<‘ or ‘>’ on the command line

Char I/O

```cpp
#include <string>
#include <iostream>
#include <fstream>   // needed for ifstream, ofstream

using namespace std;

int main() {
    char ch;
    string source, target;

    cout << "Source filename: ";
    cin >> source;

    cout << "Target filename: ";
    cin >> target;

    ifstream infile(source);   // open source file for reading
    ofstream outfile(target);  // open target file for writing

    while (!infile.eof()) {
        infile.get(ch);        // read 1 character
        cout << ch;            // display on screen
        outfile.put(ch);       // write to target file
    }

    infile.close();
    outfile.close();

    return 0;
}
```

Line I/O

```cpp
#include <fstream>
#include <string>
#include <iostream>
using namespace std;
#define MAX 200

int main() {
    string source, target;
    char str[MAX];   // buffer to hold each line

    cout << "Source file name: ";
    cin >> source;

    cout << "Target file name: ";
    cin >> target;

    ifstream infile(source);   // open source file
    ofstream outfile(target);  // open target file

    while (infile) {  // loop until file is exhausted
        infile.getline(str, n:MAX-1, delim:'\n');  // read a line (up to 199 chars or newline)
        cout << str << endl;
        outfile.write(str, n:MAX);
    }

    infile.close();
    outfile.close();

    return 0;
}
```

getline(infile, str)

Record I/O

```cpp
struct Employee {
    char name[30];
    int age;
    float sal;
};

int main() {
    Employee e;
    char ch;
    ofstream outfile( s: "EMP.DAT", mode: ios::out | ios::binary);
    do {
        cout << "Enter name age salary: ";
        cin >> e.name >> e.age >> e.sal;

        outfile.write(reinterpret_cast<const char*>(&e), n: sizeof(e));

        cout << "Add another (Y/N)? ";
        cin >> ch;
    } while (ch == 'Y' || ch == 'y');
    outfile.close();

    ifstream infile( s: "EMP.DAT", mode: ios::in | ios::binary);

    cout << "\n--- Employee Records ---\n";
    while (infile.read(reinterpret_cast<char*>(&e), n: sizeof(e))) {
        cout << e.name << '\t' << e.age << '\t' << e.sal << endl;
    }
    infile.close();

    return 0;
}
```

30 bytes	4 bytes	4 bytes

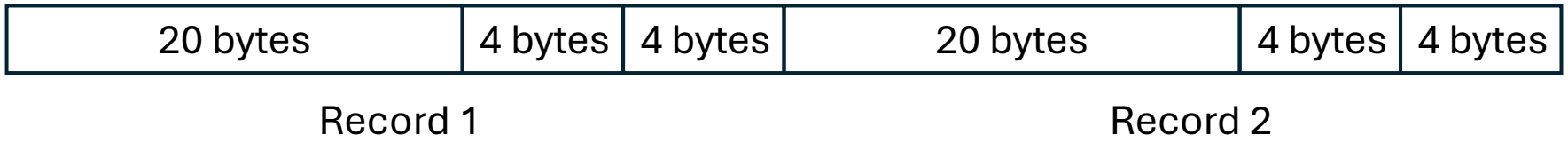
```
C:\Users\ahana\Downloads\lippman.exe
Enter name age salary:ahana 35 0
 Add another (Y/N)?N

--- Employee Records ---
ahana   35      0

Process finished with exit code 0
```

Random Access I/O

- Two pointers exist
  - get pointer – position from where next read will happen
  - put pointer – position where next write will take place
- Position get pointer
  - file.seekg (56L) ; // set at 56th byte
  - file.seekg (-28L, ios::cur) ; // other reference: beg, end
- Position put pointer
  - file.seekp (56L) ; // set at 56th byte
  - file.seekp (0L, ios::beg) ; // beg, end



File Opening Modes

Mode Bit	Result
in	Open a file for reading (default for ifstream)
out	Open a file for writing (default for ofstream)
ate	Start reading or writing at end of file (AT End)
app	Start writing at end of file (append)
trunc	Truncate file to zero length if it exists
nocreate	Error if file does not already exist
noreplace	Error when opening for output if file already exists unless ate or ape is set
binary	Open file in binary mode (not text)

C++ Exception Handling

```cpp
#include <cmath> // for sqrt() function
#include <iostream>
double mySqrt(double x)
{
    if (x < 0.0) throw "Can not take sqrt of negative number"; // throw exception of type const char*
    return std::sqrt(x);
}
int main()
{
    std::cout << "Enter a number: ";
    double x {};
    std::cin >> x;
    try // Look for exceptions that occur within try block and route to attached catch block(s)
    {
        double d = mySqrt(x);
        std::cout << "The sqrt of " << x << " is " << d << '\n';
    }
    catch (const char* exception) // catch exceptions of type const char*
    { std::cerr << "Error: " << exception << std::endl;}
    return 0;
}
```

Exceptions in C++

- Common errors found in programs
  - Compilation errors – identified by the compiler, fixed by the developer
  - Logical errors – when a program behavior is not as intended
    - identified using unit testing and integration testing
  - Run-time errors – when something unexpected occurs at run-time, e.g.
    - memory allocation results in a failure
    - a pointer is found to point to non-existent object or contains a wrong address
    - a function in a call chain finds unexpected parameters or return value
- Possible choices on handling run-time errors
  - Crash the program
  - Inform the user and exit gracefully or allow the user to recover from error
  - Take corrective action and continue running (aka handling exceptions)

Throwing an Exception

- Exceptions are generated by a throw statement
  - contains the keyword throw followed by an expression that produces a value

```
class Object {
    std::string d_name;

public:
    Object(std::string str) : d_name( str) {}
    ~Object() {}

    void func() {
        Object stringToThrow( str: "local");
        std::cout << "Calling func of " << d_name << std::endl;
        throw stringToThrow;
    }

    void hello() {
        std::cout << "Hello by " << d_name << std::endl;
    }
};
```

Catching an Exception

- Exceptions are generated within a try block delimited by { }
- One or more catch clauses must be defined after a try-block
  - Each catch clause defines a type of exception it can catch followed by
  - An action within a compound statement { } stating what to do when the defined exception is caught

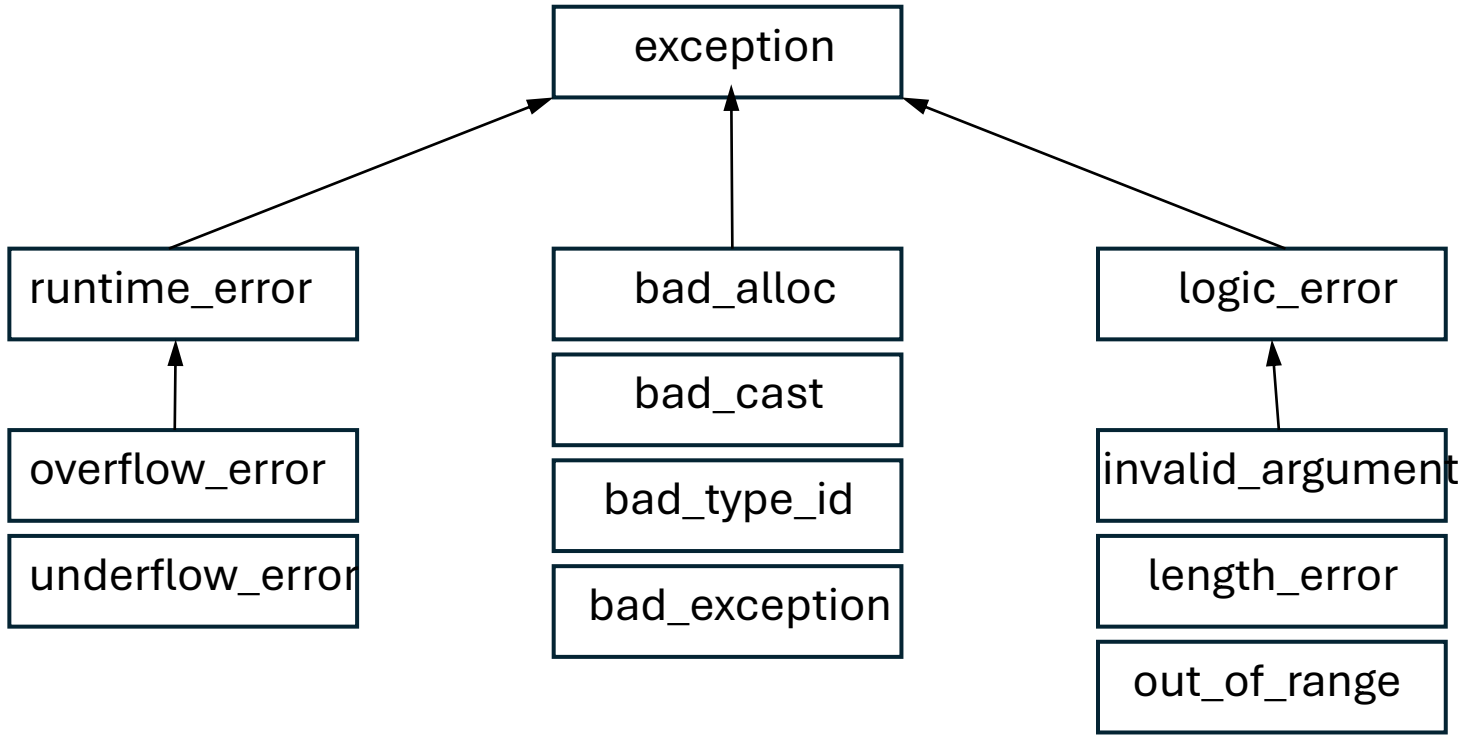
```
int main() {
    Object out( str: "main object");

    try {
        out.func();
    }
    catch (Object o) {
        std::cout << "Caught exception" << std::endl;
        o.hello();
    }

    return 0;
}
```

Calling fun of main object
Caught exception
Hello by local

Exception Classes in STL



Catching Multiple Exceptions

```cpp
int main() {
    try {
        ifstream infile( s: "Sample.cpp");
        if (!infile)
            throw runtime_error("failed to open Sample.cpp");

        string str1( s: "Virat");
        string str2( s: "Kohli");

        // Append 3 characters starting from index 7
        str1.append(str2, pos: 7, n: 3);

        cout << str1 << endl;
    }
    catch (logic_error &le) {
        cout << le.what() << endl;
    }
    catch (runtime_error &rte) {
        cout << rte.what() << endl;
    }

    return 0;
}
```

Summary of Exceptions

- A `try` block encloses any program that is likely to encounter errors
- These errors are notified by `throw`-ing exceptions, one for each kind of error
- `catch` clauses catch the exceptions thrown by the `try` block so the user can take specific actions on encountering each exception
- Pre-defined exceptions in STL may be used with `#include <exception>`
- Cleanup of resources (e.g. open files) may be required if an exception is thrown
- Unhandled exceptions cause a program to terminate